



NexScript Lab: Unlock coins, build an escrow smart contract, and compare to Ethereum

March 2025

Name:

Objective:

Unlock coins from a Nexa faucet, then spend them to deploy and run a Nexa blockchain NexScript smart contract exercises using the NexScript smart contract playground website and web wallet. Reflect on how NexScript smart contracts compare to Ethereum's smart contract model.

In this lab, you'll explore NexScript, a high-level language for smart contracts on the Nexa blockchain.

You'll unlock Nexa coins from a faucet, then spend some Nexa coins to pay for transaction fees when deploying and interacting with the NexScript smart contracts in the exercises. Finally, compare Nexa NexScript smart contracts versus Ethereum Solidity smart contracts.

Objectives

- Understand NexScript syntax and deployment.
- Build and deploy a smart contract on Nexa.
- Use the deployed smart contract.
- Compare Nexa NexScript smart contracts to Ethereum Solidity smart contracts.

Why do we use smart contracts?

Smart contracts are used to automate, secure, and decentralise transactions and agreements without relying on intermediaries. They provide a self executing, trustless, immutable, binding agreement that supports secure transactions, exchange of services or other types of agreements.

Comparing NexScript to Solidity

Feature	NexScript (Nexa)	Solidity (Ethereum)
Model	UTXO L1	Account-based, L2
Model	Native tokens	Smart-contract based tokens
Execution	Script validation in tx	EVM bytecode execution
Interaction	Construct tx's to interact	Calls contract functions
Gas Fees	Tx fees only based on script execution	Gas fees per computation step
Turing Completeness	Non-turing complete	Turing complete

Exercise Explained: Transfer With Timeout Escrow Smart Contract

One interesting use case of Nexa is using it for paper tips. With paper tips, you send a small amount of money to an address, and print the corresponding private key on paper. Then you can hand out these pieces of paper as a tip or gift to people in person. In practice, however, people might not know what to do with these gifts or they might lose or forget about them.

Alternatively, a Nexscript smart contract can be used for these kinds of gifts. This allows for leveraging a decentralized Escrow functionality, where the recipient can claim their gift anytime. Still, the sender can reclaim it if they don't claim it in time, for example, within two weeks.

Notes:

Preliminary Task: Use the Faucet

Nexa developers have a digital faucet for easily acquiring small amounts of free Nexa coins. For this computer lab, the Nexa team has deployed a mainnet version of the Nexa faucet to a secret URL, which will be used at the students' discretion at the University of Exeter.

1. Open the NexScript Playground <https://playground.nexscript.org> in your browser.
2. Access the built-in wallet within the NexScript Playground by clicking on the "Wallets" tab in the top navigation bar.
3. In the "Wallets" tab, locate your wallet (e.g., "Sender Wallet") and copy the wallet address displayed (e.g., "nexa:<address>").
4. Open the Nexa faucet at the secret URL <http://173.249.37.34:7997/faucet> and scroll down to the "Direct Address Funding" section.
5. Paste the copied wallet address into the "Enter Nexa address" field and click the "Send Funds" button.



6. Click the "Send Funds" button to tap a few Nexa from the faucet into your Nexscript Playground wallet.
7. Return to the "Wallets" tab in the Playground, refresh the wallet balance (click "refresh" if needed), and verify that you have received 100 Nexa.

Optionally, you can view the transaction on the Nexa mainnet block explorer by entering your wallet address at <https://explorer.nexa.org/> to confirm the funding. Note that 100 Nexa is a small amount and likely sufficient only for basic transaction fees in this lab. If additional funds are needed, repeat the faucet request process.

Let's Begin: Navigate to NexScript Playground

1. **Go to the NexScript Playground** (<https://playground.nexscript.org/>) and navigate to the **Edit** page. The **TransferWithTimeout contract** should already be loaded by default in the editor.

For reference, you can verify that the contract code matches the following:

```
1  pragma nexscript >= 0.8.0;
2
3  contract TransferWithTimeout(pubkey sender, pubkey recipient, int timeout) {
4      // Require recipient's signature to match
5      function transfer(sig recipientSig) {
6          require(checkSig(recipientSig, recipient));
7      }
8
9      // Require timeout time to be reached and sender's signature to match
10     function timeout(sig senderSig) {
11         require(checkSig(senderSig, sender));
12         require(tx.time >= timeout);
13     }
14 }
```

2. **In the NexScript playground, click “Compile”** at the bottom of the edit box to compile the NexScript code into a NexScript .json artifact.

The Nexscript contract artifact for the TransferWithTimeout contract is now displayed in the right window. Click ‘load artifact’ to expand and inspect its content

Contract Artifacts Import Artifact ↕

▼ TransferWithTimeout 🗑

Last Updated

2025-03-08 18:12

Artifact Bytecode

► show full Bytecode

Compiler Version

1.0.2

Download Artifact

download JSON file 📄

Load Contract to Editor

Load Artifact

The Nexscript artifact represents a compiled NexScript contract. An artifact contains all information needed to interact with the smart contracts. Artifacts are a .json file that can be shared and used later without recompiling the contract.

Deploy your contract

3A. Deploy your TransferWithTimeout smart contract to the Nexa blockchain.

- To deploy the TransferWithTimeout contract, **navigate to the New Contract page** in the Nexscript Playground and select the TransferWithTimeout artifact from the dropdown. Set the contract name to something like MyFirstContract or your Name. Each contract must have a unique name.
- The constructor input arguments required to construct the contract are the sender public key, recipient public key, and int timeout - which we will set to 41,000.

Nexscript Playground mainnet

Editor New Contract Contracts Wallets TransactionBuilder

New Contract

Choose the contract Artifact and network to use:

Select Artifact: TransferWithTimeout

TransferWithTimeout

Contract Name: ShekTesting ← Change contract name here

Initialise contract by providing contract arguments:

pubkey sender pubkey recipient int timeout

Create

Update int timeout to 41,000 ↓

Notes:

Deploy your contract

3B. Deploy your TransferWithTimeout smart contract to the Nexa blockchain, continued...

- To create a public key for the sender and the recipient, navigate to the Wallets page. Label the default wallet “Sender” and add another wallet using the “Add” button and label the second wallet “Recipient” such as:

Wallets Add Private key WIF Import

Sender

Pubkey hex:
02a7e0a843b6ed62914d3a63b0a29747d5b608d4d9a070571c7b15d96066ab039e
Pubkeyhash hex:
b5773756d5ec1454b1ca3ee21f688348bd8f2bba
Address:
nexa:nqtsq5g55gxn9g4e6vtdqpd8ex9wx36j5q6rk2c7eq9jfd24
Wallet utxos
4 utxos refresh
Wallet Balance
40000 satoshis
400 NEX
Private Key
▶ Show Private Key
▶ Show utxos



Recipient

Pubkey hex:
03a1905317fb86a256757962caf3a3ec8326863bdf8ea83fae109206f200dc149
Pubkeyhash hex:
3abefcd740a2a16611ae6716c91d8e81d007efaf
Address:
nexa:nqtsq5g58dkvmdtce8kfn8ajlqe2rqc09g4sss820uhl400
Wallet utxos
0 utxos refresh
Wallet Balance
0 satoshis
0 NEX
Private Key
▶ Show Private Key



Notes:

Sender and Recipients Explained

Sender Wallet: The sender is the individual initiating the cryptocurrency gift or tip. This person, such as a café customer wanting to tip a barista, is responsible for creating and funding the smart contract with Nexa coins, deploying it to the blockchain, and specifying the recipient's public key and a timeout period. The sender also prepares the paper tip by printing the recipient's private key (or a QR code) along with instructions, which they physically hand over to the recipient. The sender's public key is included in the contract to allow them to reclaim the funds if the recipient fails to claim the tip before the timeout period expires, ensuring their funds are not lost if the paper tip is misplaced or forgotten.

Recipient Wallet: The recipient is intended to receive the cryptocurrency gift or tip, such as a barista receiving a tip from a café customer. The recipient is given a physical piece of paper by the sender, which contains their private key (or a QR code) and instructions on how to claim the Nexa coins locked in the smart contract. To claim the funds, the recipient uses the private key to sign a transaction that calls the contract's transfer function, transferring the funds to their wallet. The recipient must act before the timeout period specified in the contract; otherwise, the sender can reclaim the funds.

Notes:

Deploy Your Contract

4. Navigate to the Wallets page in the NexScript Playground and click the sender wallet's public key hex, to copy the sender's public key, then navigate back to the New Contract page, paste it into the *pubkey sender field*. Once more, navigate back to the Wallets page, copy the recipient public key hex, navigate to New Contract and paste it into the *pubkey recipient field*:

New Contract

Choose the contract Artifact and network to use:

Select Artifact:

TransferWithTimeout

Contract Name:

Initialise contract by providing contract arguments:

Create

Click the pink create button to deploy the smart contract to the Nexa blockchain. Then navigate to the contracts page to view the contract:

Contracts

MyFirstContract

Contract address (P2ST)
nexa:nq4sq9q2spk5cnam9z5m3fpdfmyqh6evnpg4u2c507gdfvxxhqls7ft14dc4v7y3a2717wn3c37lwa4a

Contract artifact
TransferWithTimeout

Contract arguments

Details

Contract utxos
0 utxos [refresh](#)

Total contract balance
0 satoshis

Contract size
27 bytes (max 520), 21 opcodes (max 201)



Optionally, you can also copy the contract's *Nexa:* address and search for it on our Explorer <https://explorer.nexa.org/> to verify that the smart contract has been deployed to the Nexa blockchain.

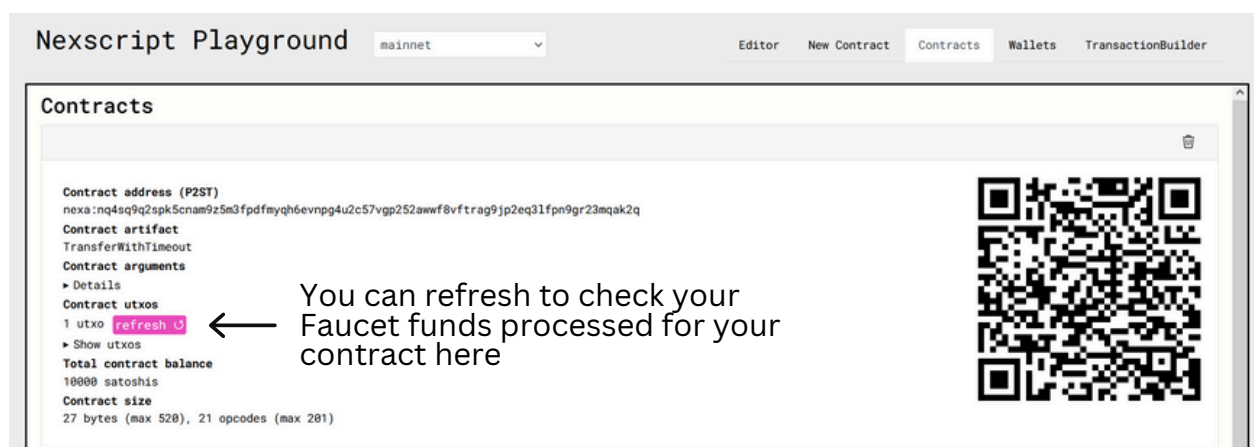
Funding and Interacting with a Smart Contract on Nexa

Step 1: Fund the Smart Contract

- The sender must now fund the smart contract, as it acts as a secure escrow for Nexa coins.
- The contract will hold the funds until either:
 - The recipient claims them, or
 - The sender reclaims them after the timeout period expires.
- When deploying the contract, the sender specifies:
 - Recipient's public key (for receiving the funds).
 - Timeout period (after which the sender can reclaim).
- After deployment, you must now send Nexa (e.g., 100 NEX) to the contract's address to lock the funds.

Task: Copy the contract address from the contracts page in the Nexscript Playground and repeat the steps from the preliminary exercise to fund the contract.

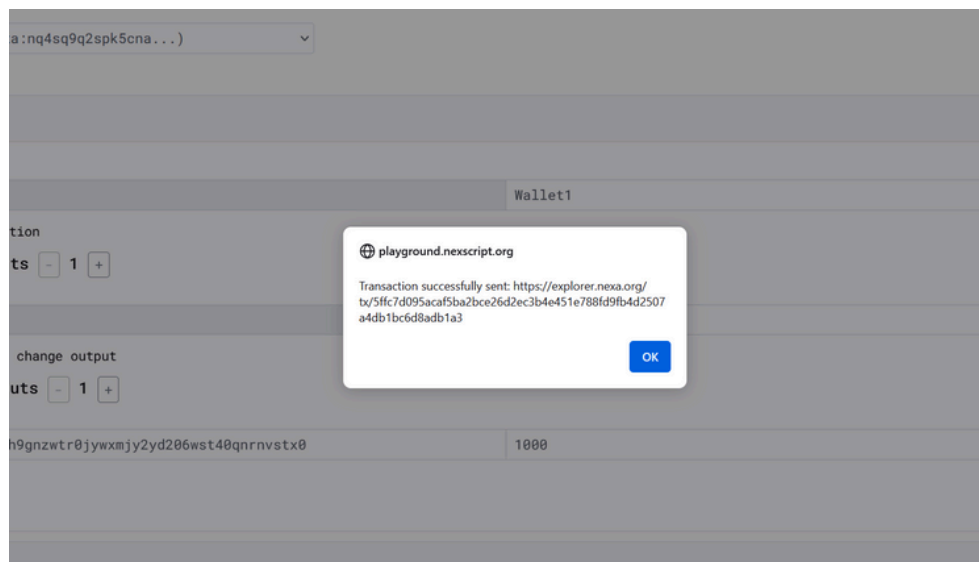
Open the Nexa faucet at <http://173.249.37.34:7997/faucet>, scroll down to the Direct Address funding input, enter the contract's address, and click the Send Funds button. Click the pink refresh button to verify that the smart contract has been funded.



Funding and Interacting with a Smart Contract on Nexa

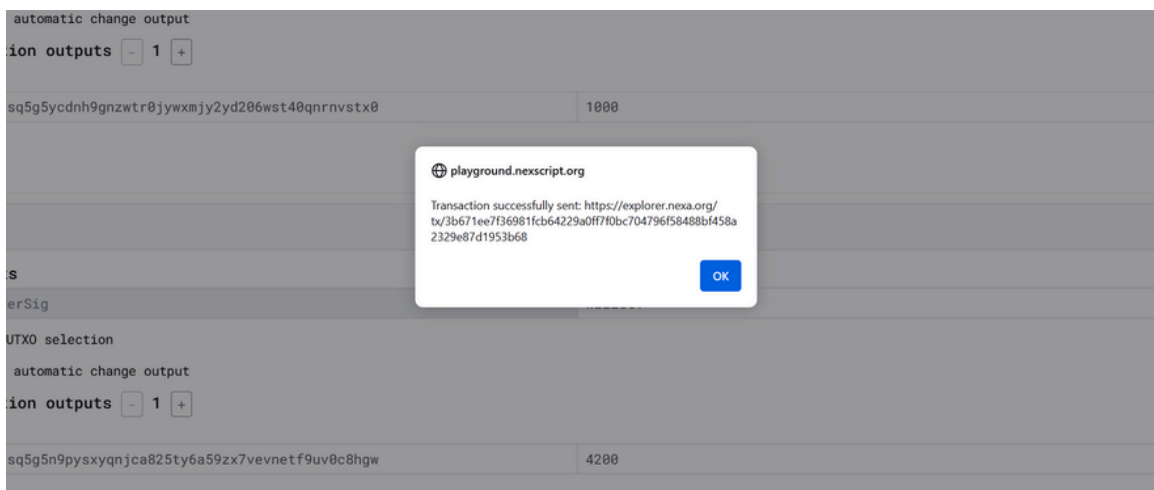
Step 2: Interacting with the Contract

1. Open the TransactionBuilder page.
2. Select MyFirstContract from the dropdown menu.
3. Set the recipient's wallet as the sig recipientSig argument.
4. Navigate to the Wallets page and copy the recipient's Nexa address.
5. Return to TransactionBuilder and:
 - Paste the recipient's address as Output #0's receiver.
 - Set 1000 satoshi (10 NEX) as the transfer amount.
 - Check the 'manual UTXO selection' box and select the contract's UTXO for which you are interacting with
6. Click the pink Send button to execute the transfer.
7. A success alert will appear with a link to the transaction in Nexa Explorer.

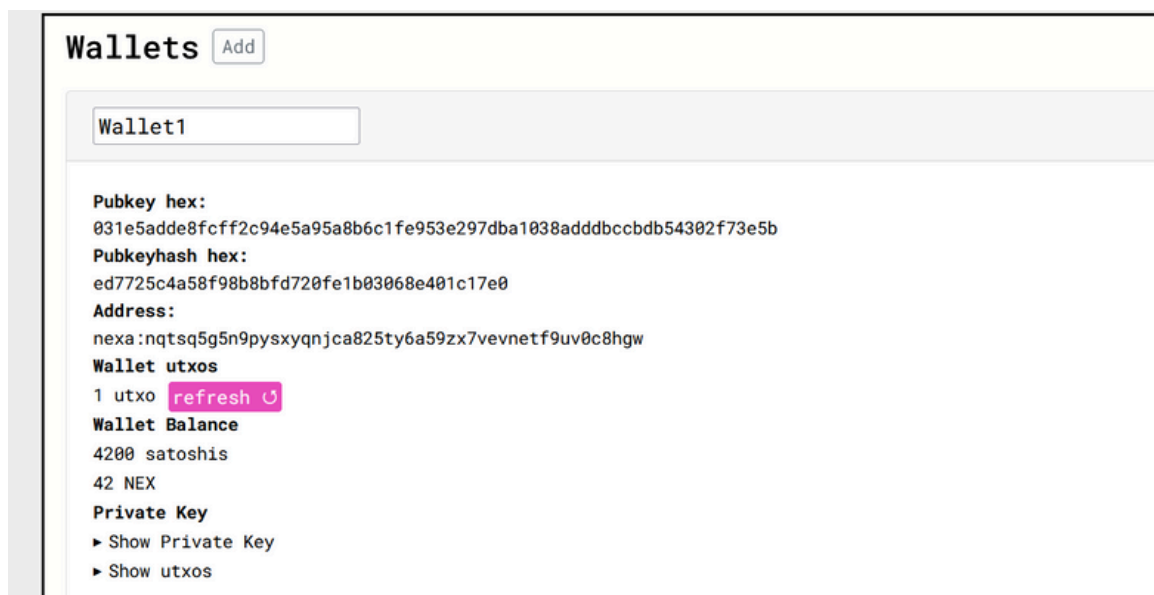


Reclaiming Funds Using the Timeout Function

1. Open TransactionBuilder and select MyFirstContract.
2. Set sender wallet as the sig senderSig argument.
3. Navigate to Wallets, copy the sender's Nexa address.
4. Return to TransactionBuilder and:
5. Paste the sender's address in Output #0 (receiver's address).
6. Set the amount to 4200 satoshi (42 NEX).
7. Click pink Send button to execute the transaction.
8. Wait for the success popup with the transaction link.



9. Go to Wallets, click the pink refresh button on the sender's wallet.
10. Verify that the 42 NEX has been successfully returned.



Congratulations! You have successfully deployed a NexScript smart contract and interacted with its functions.

Optional further work: Explore further NexScript examples found at: <https://nexscript.org/docs/language/examples>.

Optionally: Feel free to join the NexScript Telegram channel (<https://t.me/nexscript>) to ask questions directly to the Nexa team and keep exploring NexScript!

